APT: Adaptive Pruning and Tuning Pretrained Language Models for Efficient Training and Inference

Proceedings of the 41st International Conference on Machine Learning (ICML 2024), PMLR 235

Bowen Zhao (1), Hannaneh Hajishirzi (1,2), Qingqing Cao (3*)

1 University of Washington • 2 Allen Institute for AI • 3 Apple (work done at UW). Correspondence: bowen98@uw.edu, qicao@apple.com

Code https://github.com/ROIM1998/APT

- Goal: improve training and inference efficiency of LMs with minimal accuracy loss.
- Key: adaptive pruning (efficiency) + adaptive tuning (accuracy).

Figure 1 placeholder: APT adapter prunes dims/heads and grows ranks (r_apt)

×8	98%	86.4%	−70%
Faster pruning (RoBERTa/T5) vs LoRA+Prune	Task perf. with 60% pruned (small LMs)	LLaMA perf. with 70% params kept	Peak training memory (large LMs)

Introduction

- Scaling LMs improves accuracy but raises training/inference costs (e.g., LLaMA-13B: ~100GB FT, ~30GB INF @fp16).
- Parameter-Efficient Fine-Tuning (PEFT) reduces training memory but not inference cost (model size unchanged).
- Structured pruning speeds inference but increases training time/memory and hurts convergence.
- Challenge: jointly gain training + inference efficiency without large accuracy loss.

Figure placeholder: Problem landscape — Parameter-Efficient Fine-Tuning (PEFT) vs Pruning vs APT

APT: Core Idea

- Early pruning: discard task-irrelevant parameter blocks to cut compute/memory.
- Adaptive tuning: dynamically add salient adapter ranks to recover/boost performance.
- Lightweight salience scoring and efficient search enable low-cost adaptation.
- Self-distillation with shared weights improves accuracy with minimal overhead.

Figure 2 placeholder: Workflow — compute salience → prune blocks → grow ranks → distill

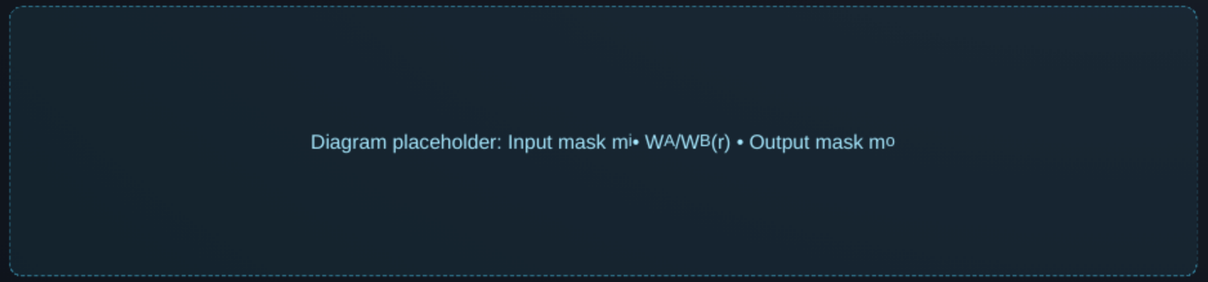
Problem Formulation

- Objective: minimize task loss with target sparsity γ_T after T steps.
- Per step constraints: sparsity $\geq \gamma_t$, tuning params $\leq \Delta_t$.
- Controls: pruning masks M_t and adapter ranks R_t.
- Trade-offs: higher γ → faster inference; larger Δ → faster convergence but more train memory.

We dynamically adjust $C(\Theta_t, M_t)$ and $\delta(\Theta_t, M_t, R_t)$ during fine-tuning.

APT Adapter

- LoRA (Low-Rank Adaptation)-based module with binary masks m_i, m_o and dynamic rank r_{apt}.
- Prunes: hidden dims (inputs), heads in MHA (Multi-Head Attention)/FFN (Feed-Forward Network) neurons (outputs).
- Placement: Q/V in MHA; FFN for smaller models.
- Ranks grow over training; masks prune blocks for efficiency.



Adaptive Pruning (AP)

Salience Scoring

- Outlier-aware: uses activation–gradient magnitudes.
- Batch-summed activations/gradients to save memory.
- Kurtosis combines to preserve outlier-heavy blocks.

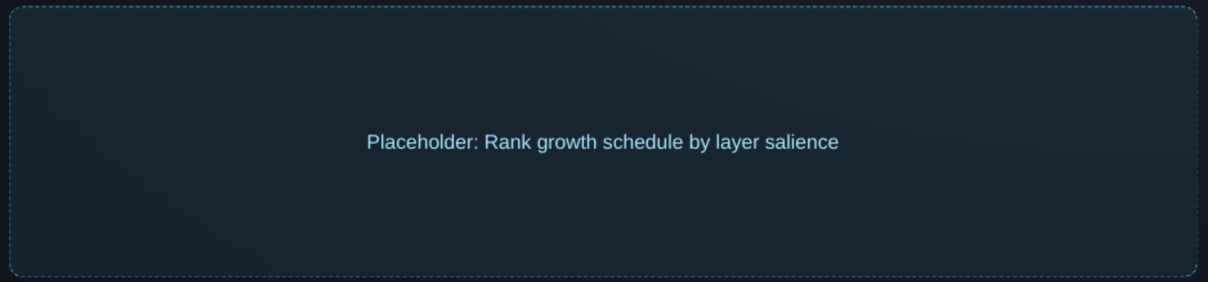
Efficient Block Search

- Score blocks by salience per parameter.
- Binary search threshold to satisfy target sparsity.
- Prune heads, FFN neurons, model dims jointly.

Pruning starts early ($t \ll T$) to cut training time/memory while maintaining stability.

Adaptive Tuning (AT)

- Compute adapter importance $I(H_{apt})$ from parameter salience.
- Increase ranks for top-half salient adapters within budget Δ_t.
- Stable growth: concat Gaussian init on W_A, zeros on W_B (LoRA-style).
- Speeds convergence; recovers pruned performance efficiently.



Efficient Self-Knowledge Distillation (DS)

- Teacher = duplicated tuning layers; shares frozen weights with student.
- Reduces memory and time vs separate teacher models.
- Objective blends supervised FT and dynamic layer-wise distillation.
- Layer mapping with block-wise sampling; transformation initialized as identity.

μ linearly increases from $0 \rightarrow 1$ during training to balance fitting and distillation.

Related Work: Parameter-Efficient Fine-Tuning (PEFT)

What is PEFT?

- Parameter-Efficient Fine-Tuning (PEFT) updates a small subset or additive modules to adapt LMs with low training memory, leaving most weights frozen; inference model size typically unchanged.

Parameter-Efficient Fine-Tuning (PEFT)

- Selective, additive (adapters/LoRA), and dynamic (AdaLoRA).
- Typically training-efficient; inference unchanged.

Compression

- Quantization: memory reduction; speed requires hw support.
- Pruning: unstructured (sparse hw), structured (heads/neurons/dims) for general speedups.

Combining PEFT + Compression

- Prior combos (SPA, CPET, PST, LRP) suffer accuracy drops.
- APT uses salience-driven pruning + dynamic tuning to mitigate losses.

Results

- Small LMs (RoBERTa/T5, 60% sparsity): retain up to 98–99% FT accuracy with up to ×8.4 faster pruning; inference up to ~2.4× faster with ~78–82% memory.
- LLaMA-2 7B (70% params kept): 86.4% average performance with ~75.8% training and ~67.2% inference memory vs LoRA; comparable step time.
- APT improves both training and inference efficiency versus LoRA+Prune, CoFi-based baselines, and LLMPruner at matched sparsities.

Conclusion

- APT couples adaptive pruning with adaptive tuning to jointly optimize training and inference.
- See Results slide for consolidated empirical outcomes; APT maintains strong accuracy under significant parameter reduction.
- Enables resource-friendly fine-tuning across small and large LMs.

Future: broader PEFT integration and improved salience/tuning strategies.

References (Selected)

- Devlin et al., 2019; Liu et al., 2019; Raffel et al., 2020; Kaplan et al., 2020.
- Hu et al., 2022 (LoRA); Zhang et al., 2023b (AdaLoRA); Pfeiffer et al., 2021.
- Xia et al., 2022 (CoFi); Kwon et al., 2022 (Mask Tuning); Lagunas et al., 2021.
- Dettmers et al., 2022; Frantar et al., 2023; Lin et al., 2023 (quantization).
- Ma et al., 2023 (LLMPruner); Zhang et al., 2023a (LRP); Li et al., 2022 (PST).
- Haidar et al., 2022; Hinton et al., 2015 (distillation); Shen et al., 2022.

See paper for full references and appendices.

Appendix: Ablation Highlights

- Adaptive Tuning (AT) restores accuracy and speeds convergence (~16% faster vs static); Adaptive Pruning (AP) crucial for LLaMA memory; removing kurtosis drops LLaMA avg from 50.0→38.1; Distillation (DS) improves accuracy with modest overhead.

One-Sentence Takeaway

APT adaptively prunes and tunes during fine-tuning to deliver up to ×8 faster training with ~70% lower memory while preserving strong task performance (see Results slide).

Appendix: Training Setup Details

- Single A100 GPU; inference batch sizes: 128 (small), 32 (LLaMA 7B), 4 (LLaMA 13B).
- Time-to-Accuracy (TTA) measured at 97% of Full Fine-Tuning (FT) accuracy; efficiency metrics normalized to FT (small) or LoRA (Low-Rank Adaptation) (LLaMA).
- LLaMA trained on Alpaca; GLUE/SQuAD/CNN-DM per paper settings.

Appendix: APT Adapter Placement

- Inserted into Q/V projections of MHA (Multi-Head Attention) for all backbones.
- FFN (Feed-Forward Network) adapters additionally used for smaller LMs (RoBERTa/T5) to speed convergence.
- Masks m_i prune hidden dims; m_o prune heads/neurons.

Appendix: Salience Scoring Details

- Activation–gradient product used due to frozen-weight gradients in Parameter-Efficient Fine-Tuning (PEFT).
- Batch-sum compression before product lowers memory overhead.
- Kurtosis augments block scores to preserve outlier-heavy structures.

Appendix: Block Search Procedure

- Sort blocks by salience per parameter to estimate benefit per cost.
- Binary search threshold to meet target sparsity γ_t.
- Jointly reduce heads, FFN neurons, and hidden dims.

Appendix: Adaptive Tuning Mechanics

- Adapter importance $I(H_{apt})$ = sum of salience over W_B parameters.
- Increase ranks for top-half salient adapters proportionally to Δ growth.
- LoRA (Low-Rank Adaptation)-style init keeps pre-growth function unchanged.

Appendix: Self-Distillation Objective

- Shared frozen weights; duplicated tuning layers as teacher.
- μ schedules $0 \rightarrow 1$ to balance L_{ft} and $L_{distill}$.
- Teacher layers block-wise sampled; mapping to closest non-pruned student layer.

Appendix: Additional Ablations

- RoBERTa: w/o Adaptive Tuning (AT) ≈ LoRA+Prune accuracy; AT improves convergence by ~16%.
- LLaMA: removing kurtosis in salience drops avg from 50.0 to 38.1 (30% sparsity).
- Removing Distillation (DS) saves time/mem but reduces accuracy (~1.35 avg).

Appendix: Efficiency Metrics Normalization

- Training speed reported as time per step (LLaMA) or Time-to-Accuracy (TTA) (small models).
- Inference speed via throughput; memory via peak allocated GPU memory.
- RoBERTa/T5 normalized to Full Fine-Tuning (FT); LLaMA to LoRA (Low-Rank Adaptation) baseline.

Appendix: Discussion & Limitations

- Pruned LLaMA gap larger due to distillation-free setup; future: memory-efficient distillation.
- Hardware-friendly shapes (e.g., multiples of 8/16) can yield better real speedups.
- Dynamic parameter changes may introduce instability; optimizer resets help.
- Explore richer adapters (prefix, parallel) while preserving inference efficiency.

Appendix: Acronyms and Definitions

- PEFT — Parameter-Efficient Fine-Tuning: adapts LMs by updating a small set/additive modules; inference size unchanged.
- AP — Adaptive Pruning: early, salience-based removal of heads/neurons/dims to cut compute and memory.
- AT — Adaptive Tuning: dynamically grows adapter ranks in salient layers within a budget.
- DS — (Self-)Knowledge Distillation: teacher–student training with shared frozen weights to recover accuracy.
- LoRA — Low-Rank Adaptation: additive low-rank matrices enabling memory-efficient fine-tuning.
- FFN — Feed-Forward Network: transformer MLP sublayer whose neurons can be pruned.
- MHA — Multi-Head Attention: transformer attention with multiple heads that can be pruned.